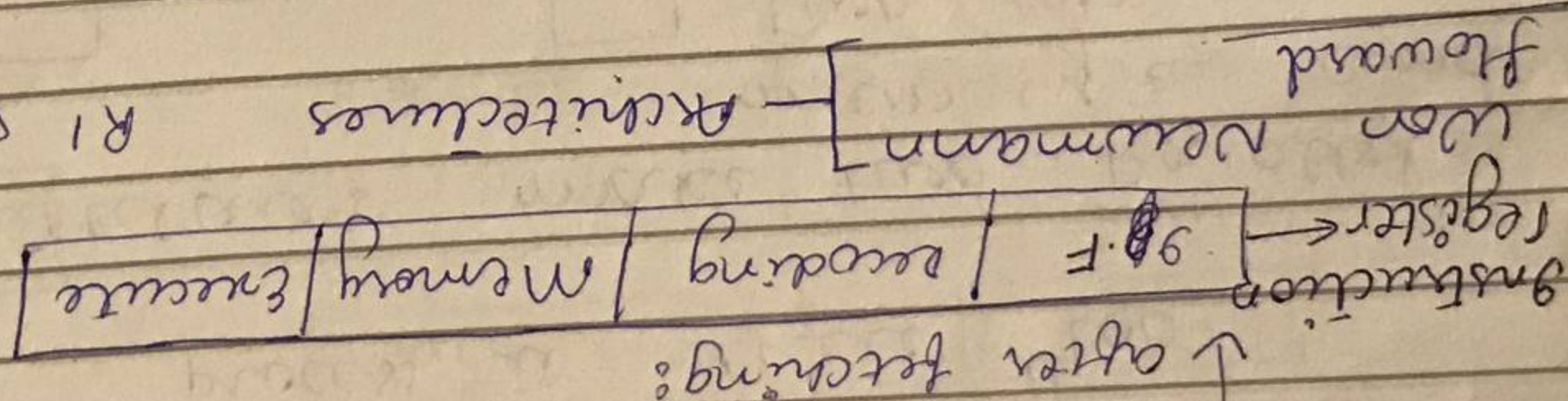
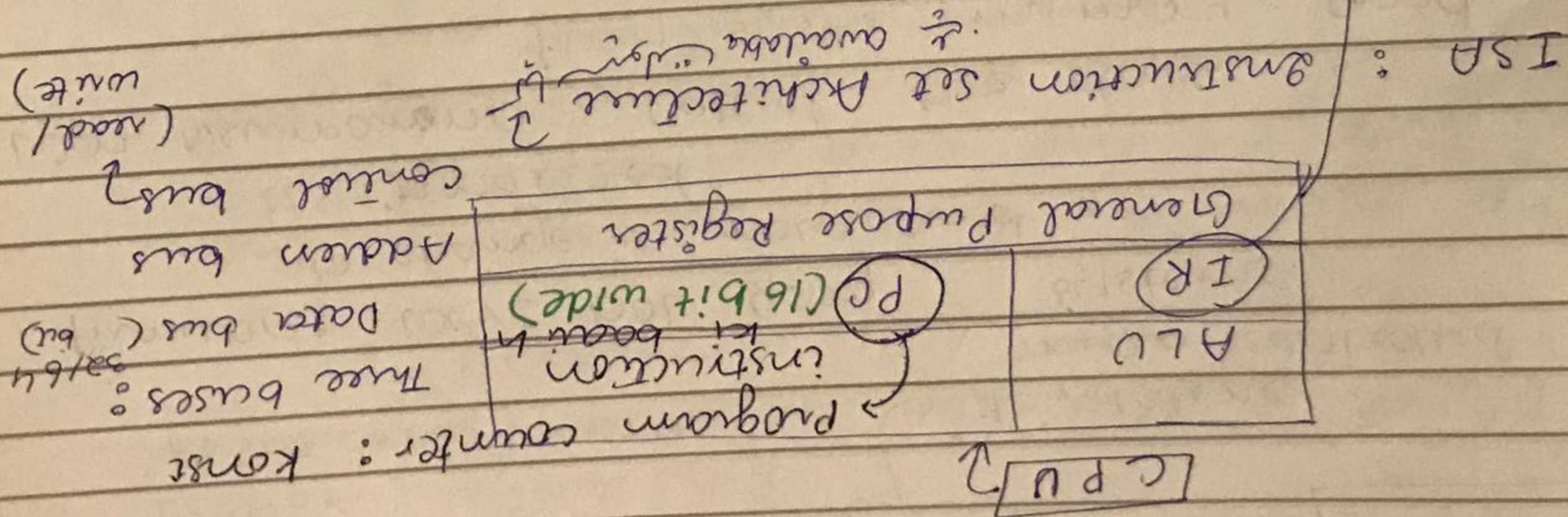


1 KByte = 2^{10} bytes
1 MByte = 2^{20} bytes

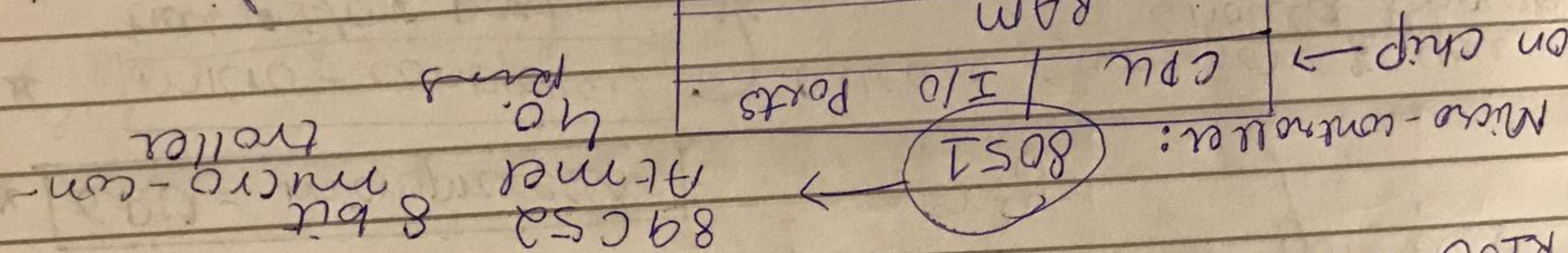
• Bohat zeda meend ai hai my:



von Neumann Architecture $R1 \leftarrow (R1 + R2)$

CISC: size of the fetched data can vary. complex instruction set construction.

RISC: Reduced Instruction set construction.



Idle dead time hi mai gura rahu.

2

any MC system that performs a specific task and cannot be modified easily

• Software of embedded system is called firmware

• RESET mechanism

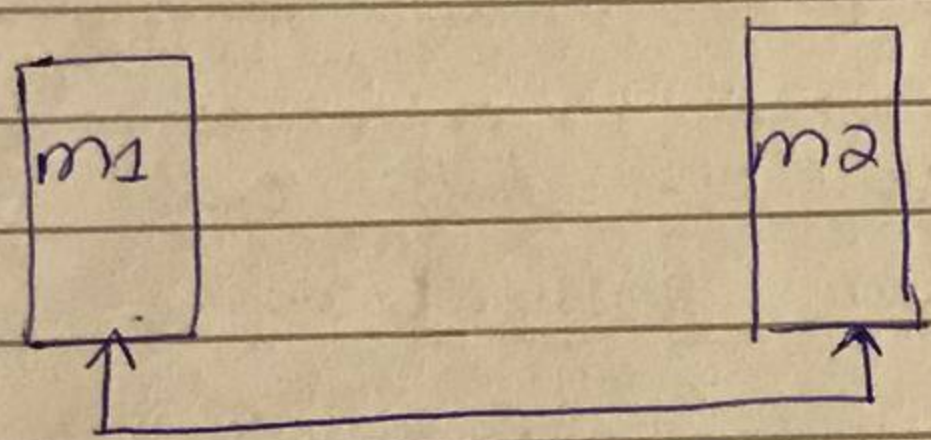
* standalone micro-controlled systems.

firmware: we're not going to change it now and its finalized.

von Neumann: collect the data from same memory and process it there too.

Harvard Architecture: make two parallel memories and two sets of busses to make the process faster. one for storage information and other for data only.

[(ku)
Fetch hotay
sth dusri
instruction
pay kaam
karna shuru
kardega.



* CPU = microprocessor

* micro-controller has • I/O parts,

atleast functions you should know about MC:

- memory
 - ROM (size?)
 - RAM (size?)
- Timers/counters
- Reset circuitary *
- ①* → Data size
- Interrupt structure

- Ad-on features:
- UART on chip
 - PWM
 - ADC
 - Debugging feature
 - flexibility.

③

21-08-2017

MC

Monday
8051 Atmel $\xrightarrow{\mu}$ 8-bit dataC $\xrightarrow{\mu}$ 89C52 $\rightarrow$ 89C51
EEPROM $\rightarrow$ electrically erasable Programmable
ROM. (Flash memory)

* (To burn the binary format of the
code on the chip $\rightarrow$ USB ports
, its better than Serial configuration.

40 pins IC

89C51

89C52

freq:

11.0593MHz

4 I/O Ports

4 I/O Ports

(each port has 8 pins/bits) $\rightarrow$ same

Byte & bit addressable — same

RAM: 128 bytes

256 bytes

ROM: 4 kbytes

8k bytes

Timers: 2

3

Interrupt sources: 6

8

UART: 1

1

P0 $\rightarrow$ Pin '0' $\rightarrow$ P0.7

Port I/O

P0

P1

P2

P3

MSB

P2.7

LSB

P2.0

Pin number
bit numbers

P0.0

Least significant bit

of Pin 0

$\rightarrow$ P0.0

MSB of pin 0 $\rightarrow$ P0.7

(24)

[P0] → whole ~~bit~~ byte

P0.7 → bit accessible

[I/O]

40 pins (total)

P0
P1
P2
P3

each has 8 pins

$$= 8 \times 4 = 32 \text{ pins}$$

• 32 pins I/O

• 2 for VCC

• 1 Reset Circuitry

8 bits → each location is 8 bits wide.

89C52

89C51

RAM:

$$256 = 2^8 \text{ bytes}$$

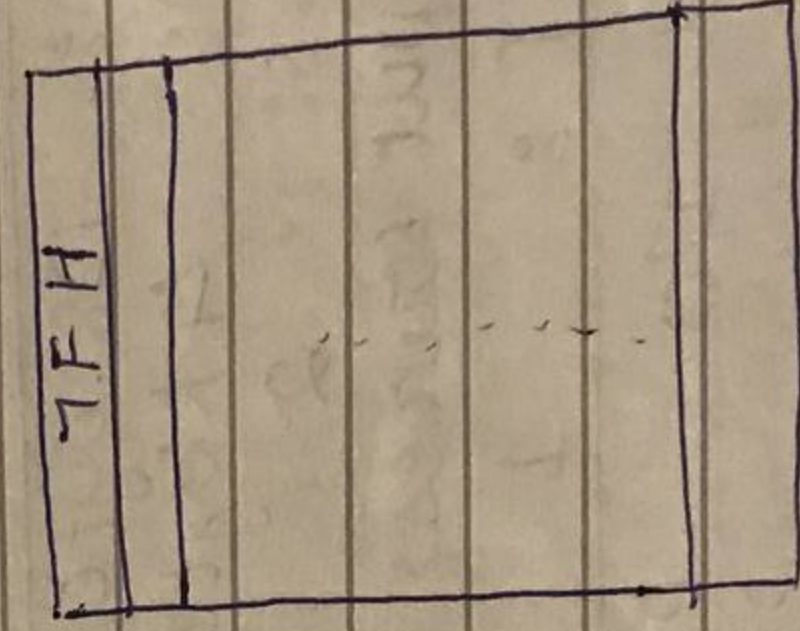
$$128 \text{ bytes} = 2^7$$

at least 7 bits

are required to access

a time where each

is carrying 8 bits



* ROM: 4 k Bytes

$$4 \times 2^{10} \text{ bytes}$$

$$2^2 \times 2^{10}$$

$$2^{12}$$

bytes → we need 12 bits to access each byte in 4kbyte ROM.

⑤

Hexadecimal 0-255

e.g. $2^n = 255$

1111 1111
7654 3210

OFH

↓ immediate next location ↓ 1 AH

10H

1 B H

1 9 H
1 A H

Poling: continuously monitoring the process of pin.

Bit operations:

✓ clr A

x setb A

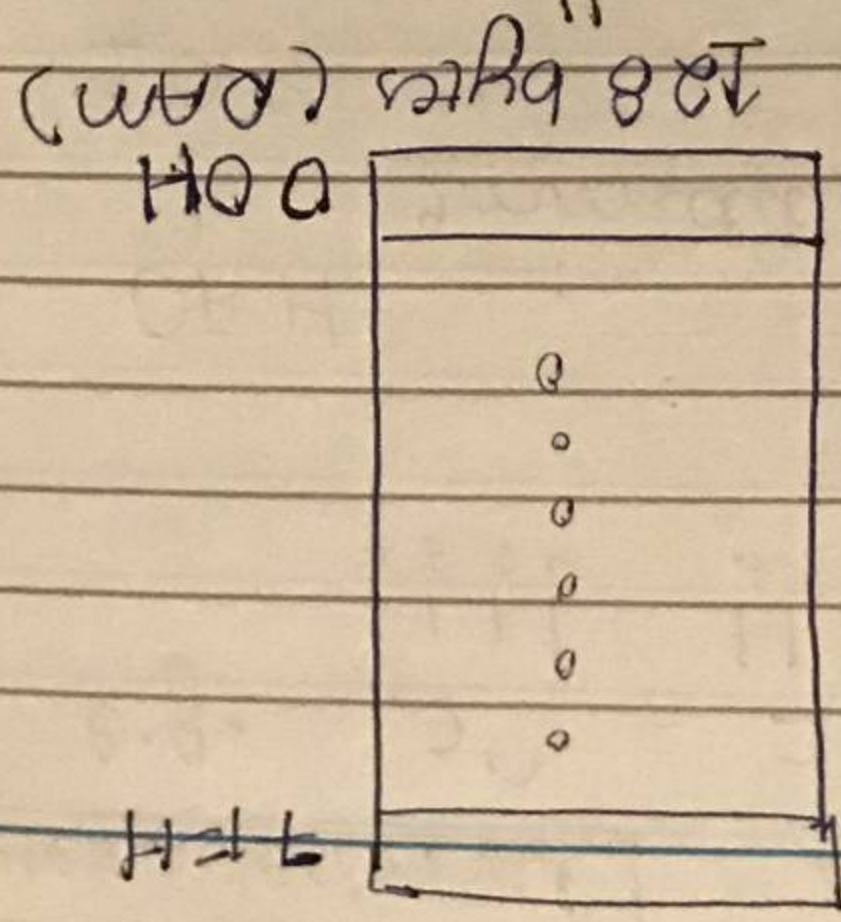
✓ cpl A

Bit Accessible
 Byte Accessible

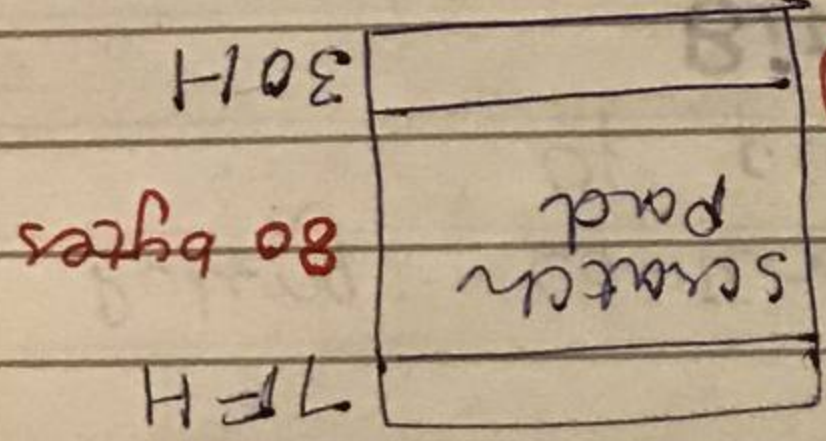
Section 1

Addressing modes:

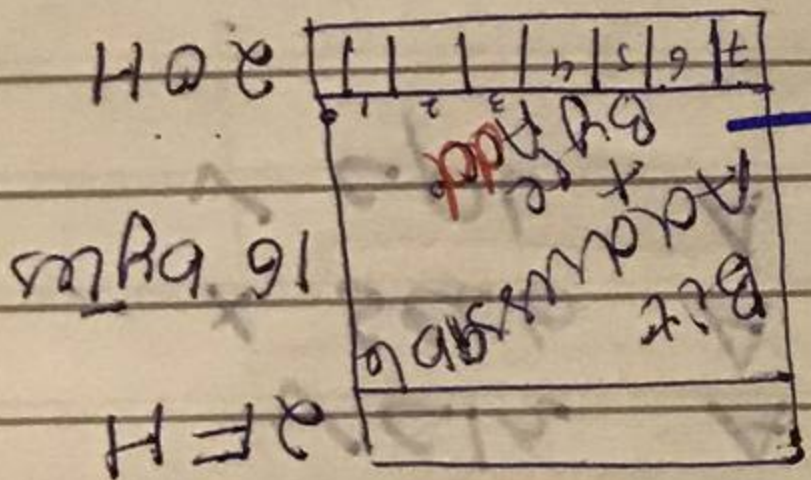
- Direct
- Indirect
- Index
- Registers



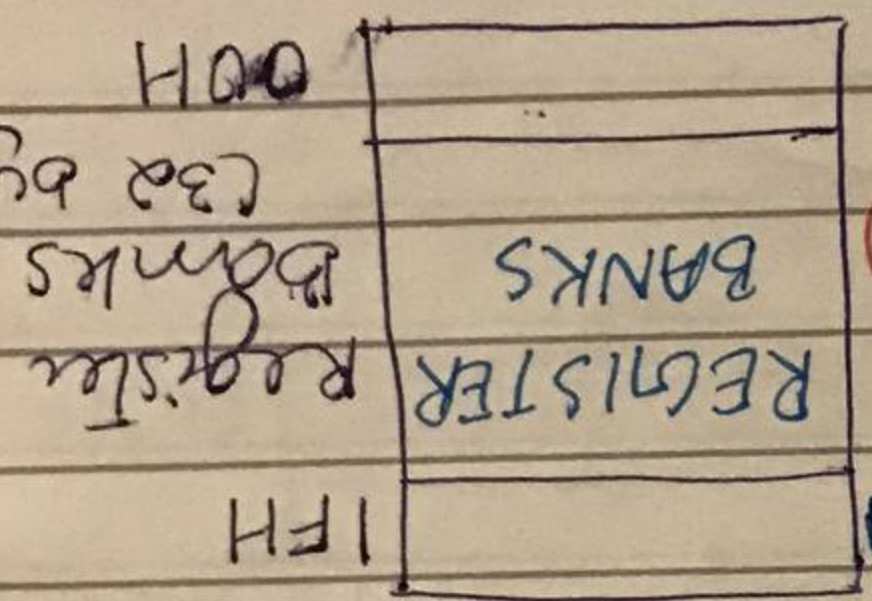
only using location



(General Purpose)
 only byte addressable

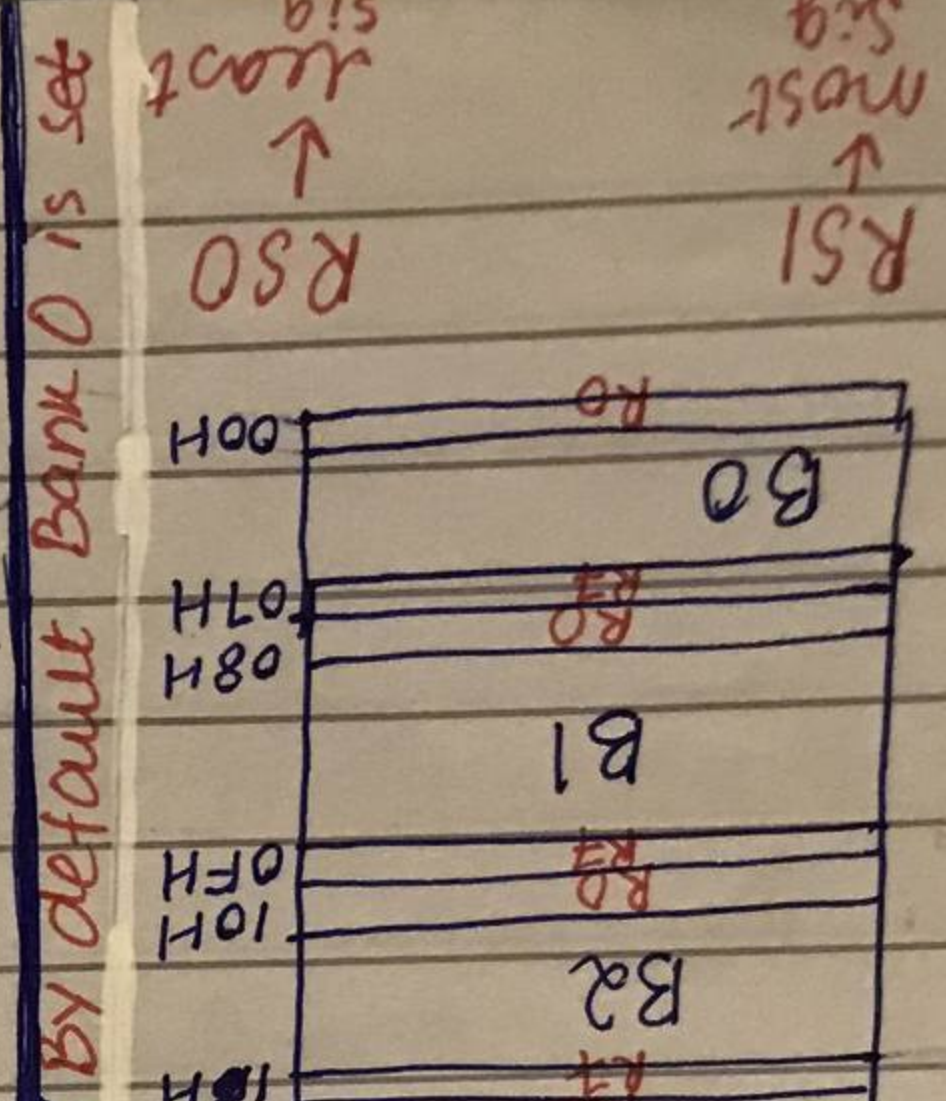
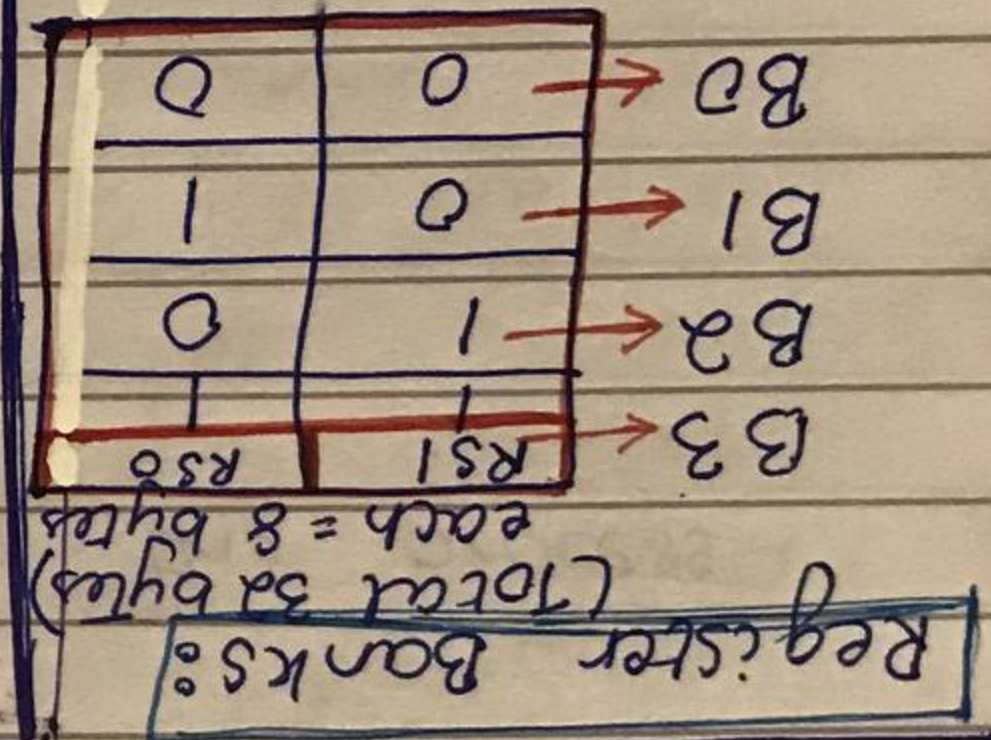


Positions of RAM:
 e.g. 20H, #23H

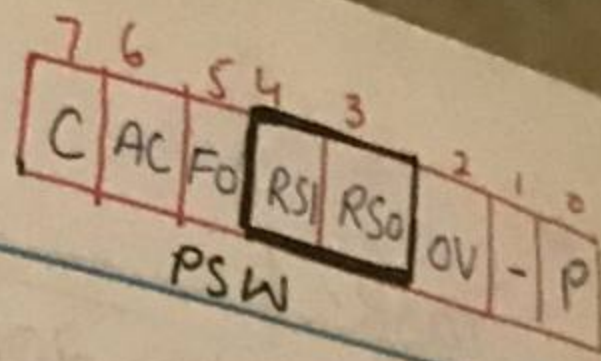


can be addressed
 by register

Positions of RAM
 are parts of



PSW → Program Status Word
 Flag register in MC



output
 setb RSI → setting 1 to pin RSI
 setb RSO
 clb RSI → setting 0 to pin RSI
 cpl RSI → read modify write (toggle the status of pin)
 setb PI.0

Data Transfer Instructions

no decimal with number → Decimal

mov R0, #01
 ↳ to take any number from the user.
 Immediate Addressing mode.
 (0 - 255) can be stored in R0
 • If want to store a number greater than 255
 Divide it into 2 parts → upper lower

mov R0, #10H
 hexa.

mov R0, #0001 0000 (B) → Binary

Task 1:

Decimal

store 201 in R4 of Bank 3.

Sol: $\rightarrow$ bit operation

setb R51 or set PSW.4

setb R50 or set PSW.3

mov R4, #201

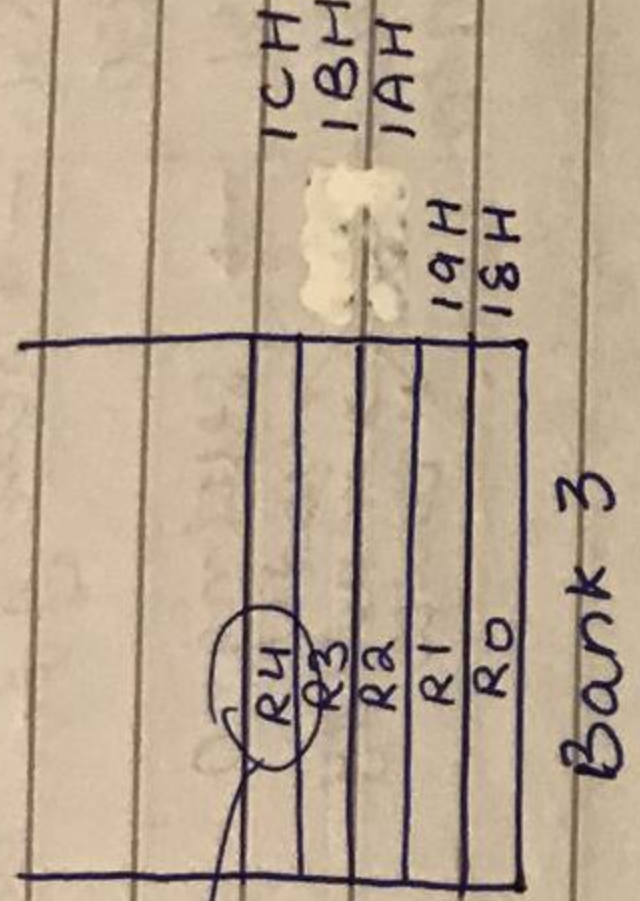
put 201

(in hexa)

here

that is

C9H



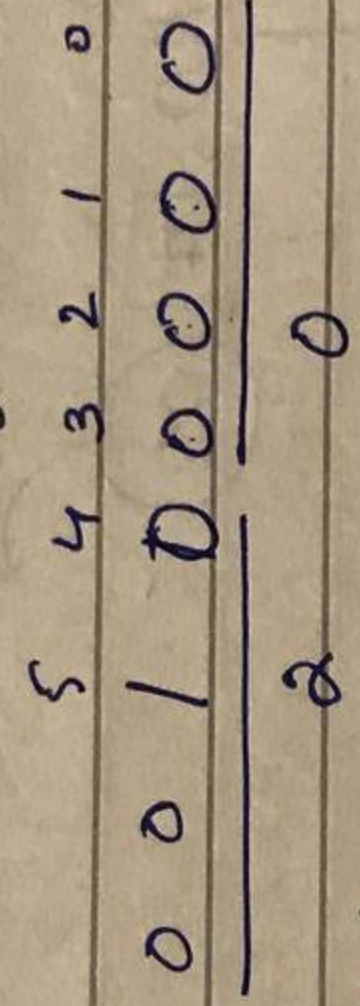
Task 2:

Store 32 Decimal in R6 of B2

$\rightarrow$ bit operation

clr R50

setb R51



immediate

mov R6, #32 $\rightarrow$ addressing mode $32 = 2^5$

$$32^D = 20H$$

28-08-2017

①

Immediate Addressing mode (mov) $\rightarrow$ The operand

ISA of 89C51 doesn't apply is immediately available

multiple addressing

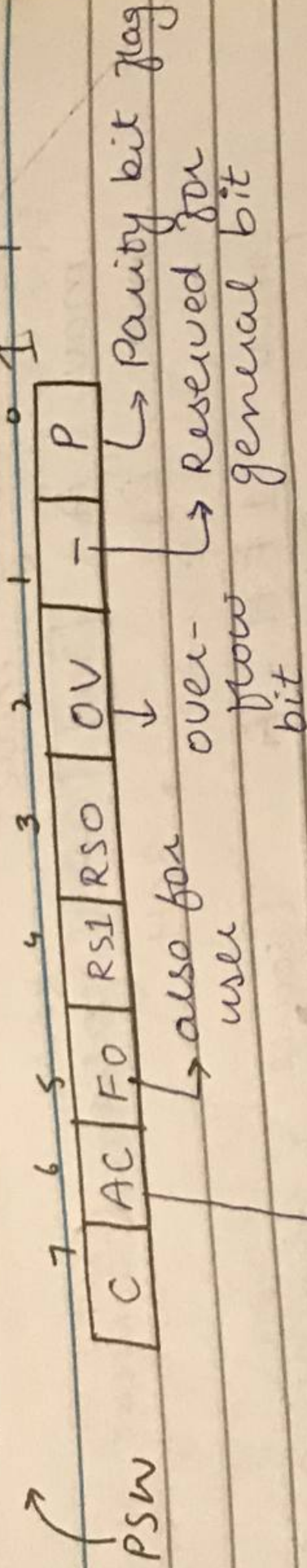
1 source 1 destination

e.g. mov R0, #30H

source.

(9)

8 bits



Auxiliary carry

Register Addressing mode:

List in registers:

RD - R7

- Accumulator A has the size of 8 bits
- Destination register having size of 8 bits

Q. Add R1 and R0 and move to R2.

mov A, R0

add A, R1

mov R2, A

Def:

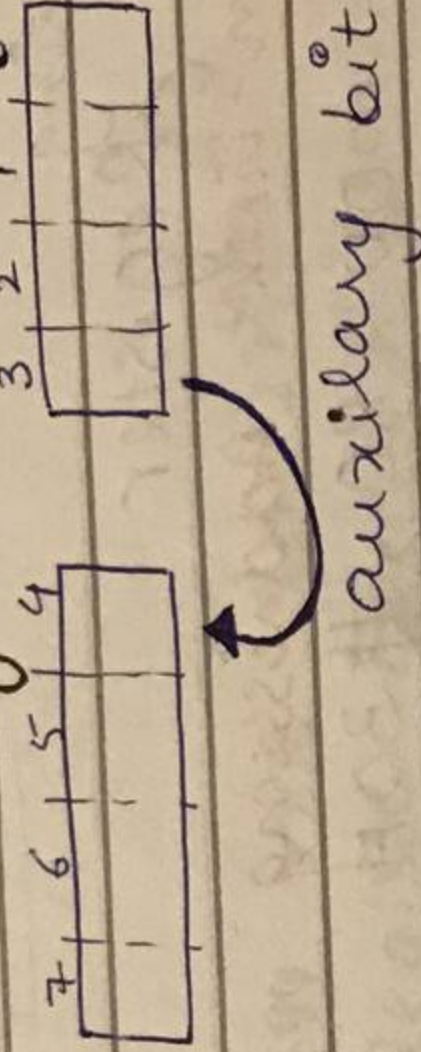
"If the source and destination both are registers, then the addressing mode we are using is Reg. addressing mode"

mov R0, R1

mov Ri, Ri
i = 0, 1, ..., 7

DLD Revision

1 byte = 2 nibbles



+2 0000 0010 B
-2 1111 1110 B
F E H

mov R4, #-2
it will store '-2' by first converting it into 8's comp. then its Hexa.

(10)

(3) Direct Addressing mode:

RAM
MOV A, [30H] → location (copy the content of 30H in A)

MOV Ri, 0 → 7, 30H

MOV 1FH, A (source & destination) or either 1 of the 2.

[MOV 31H, 30H] → Both sides can be mentioned as direct address.

→ Not allowed in Register addressing mode.

using Register

(4) Indirect Addressing mode: → can use contiguous RAM locations during Runtime

{ MOV R0, #30H

MOV @R0, #23

→ Treat the value of R0 as ADDRESS

→ [23]

30H (location)

* CONDITIONAL JUMP*

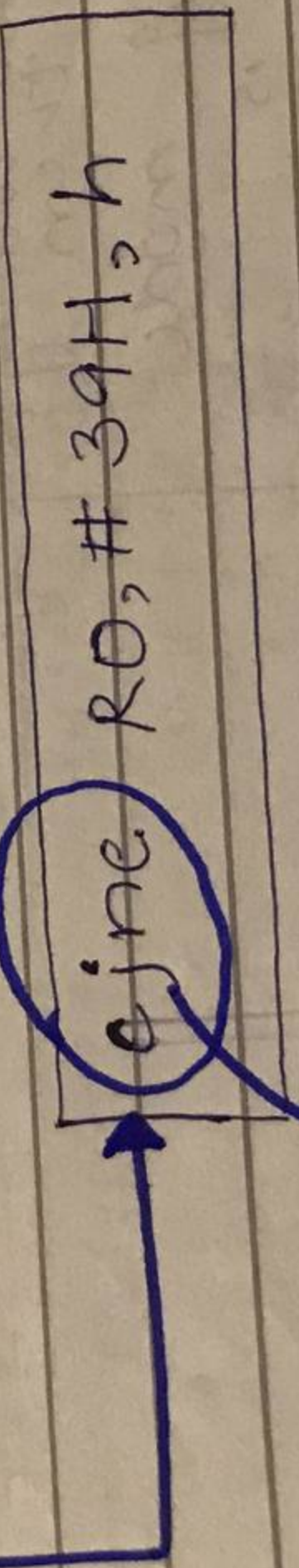
e.g.

Write #23 in 30H → 39H

MOV R0, #30H

h: → MOV @R0, #23

inc R0

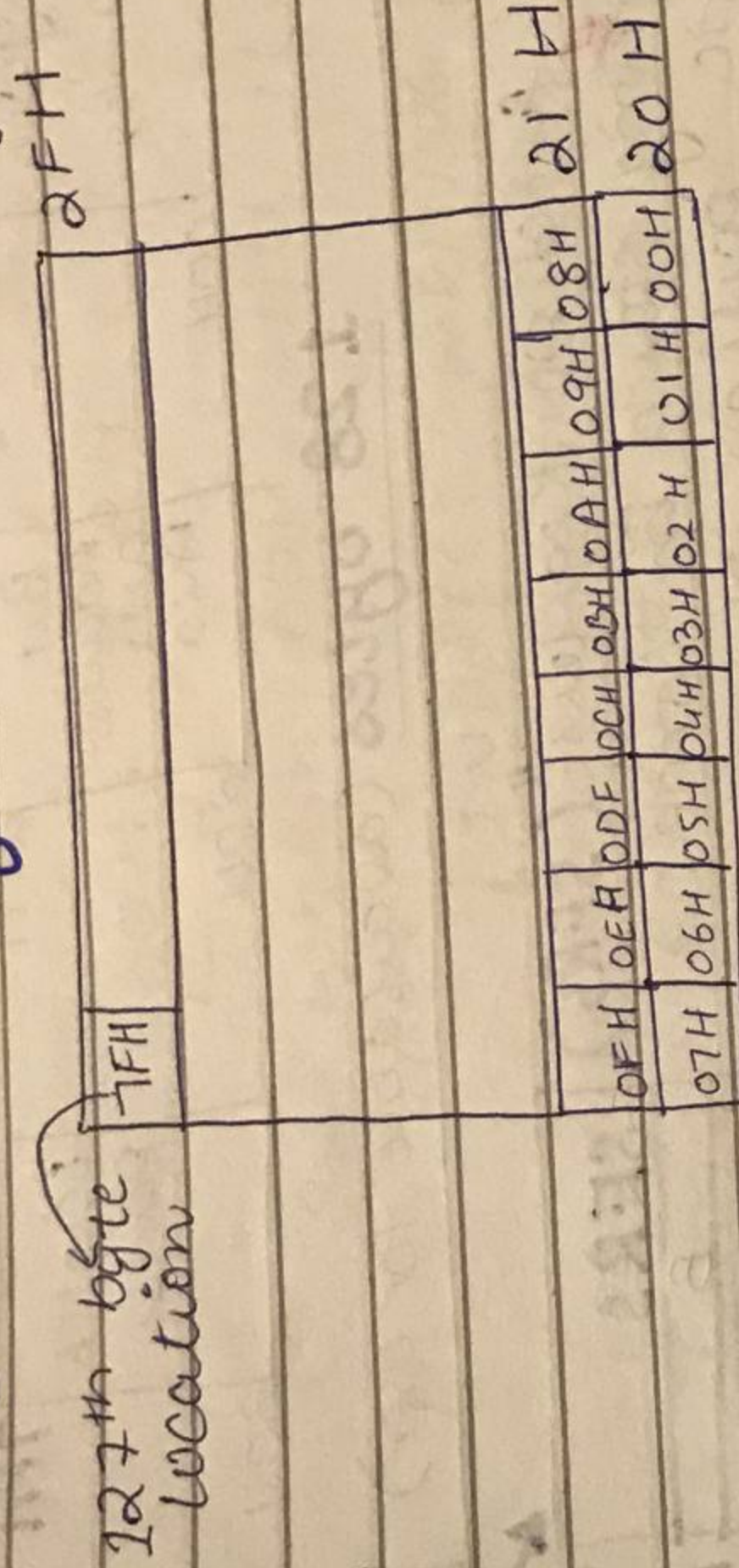


for comparing values
cjne: compare jump not equal.

(11)

and Portion of RAM:

Bit addressable,
byte



MOV C, 00H

will not go into Register Bank # '0' because Destination C' is a bit.

LSB

MOV 0H, #12H

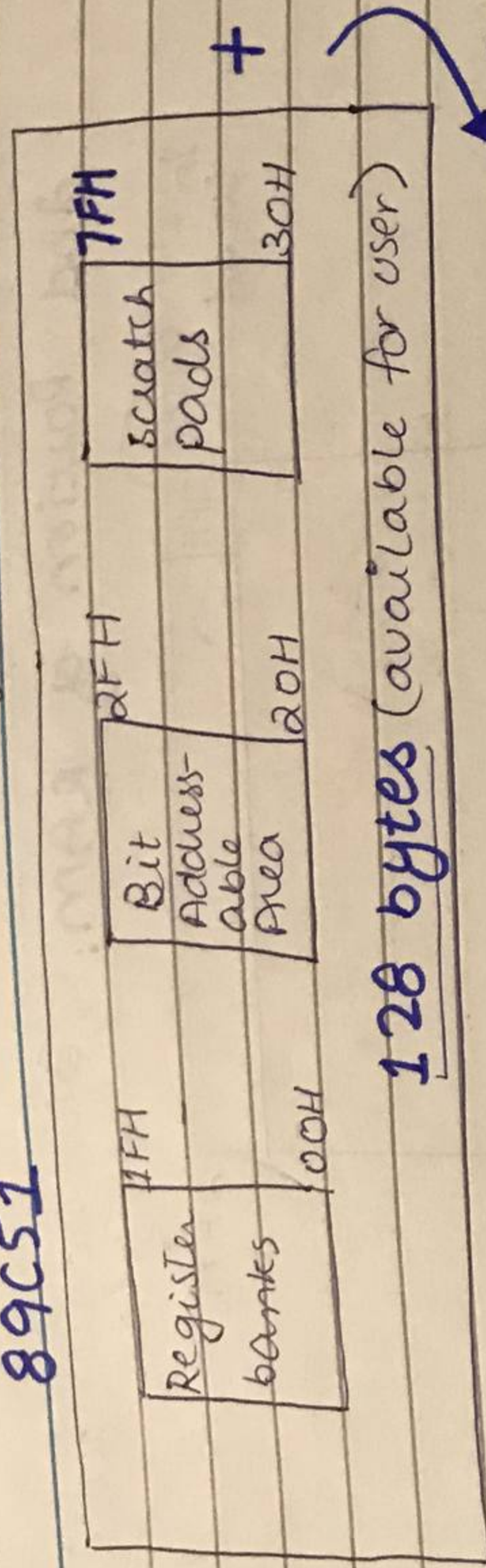
setb 00H → will set 00H of 20H in and portion of RAM.

Scratch Pad is byte addressable and there's no other alternate name for it.

(12)

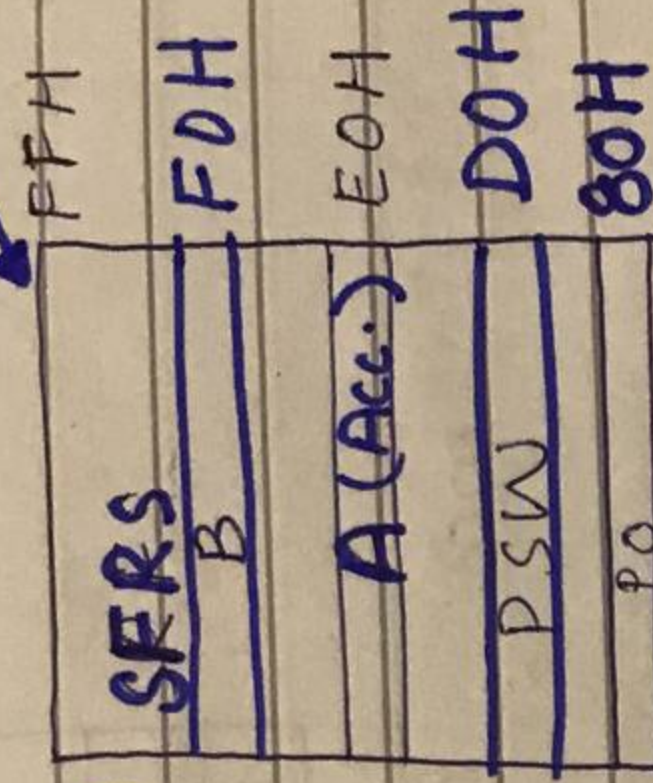
→ actually 256 bytes but only 128 bytes available for user.

89C51



"Special function Registers" (SFRs)

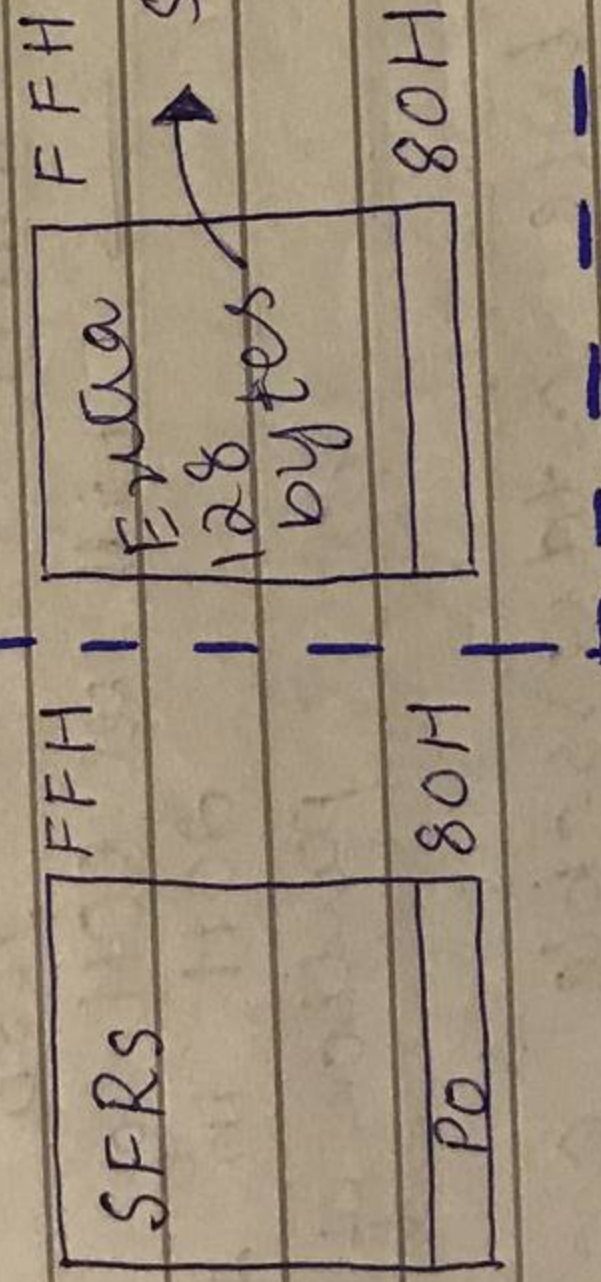
are being stored in some specific portion of our RAM.



89C52 → First 128 bytes of 89C51 +

→ can be accessed using

INDIRECT ADDRESSING MODE



256

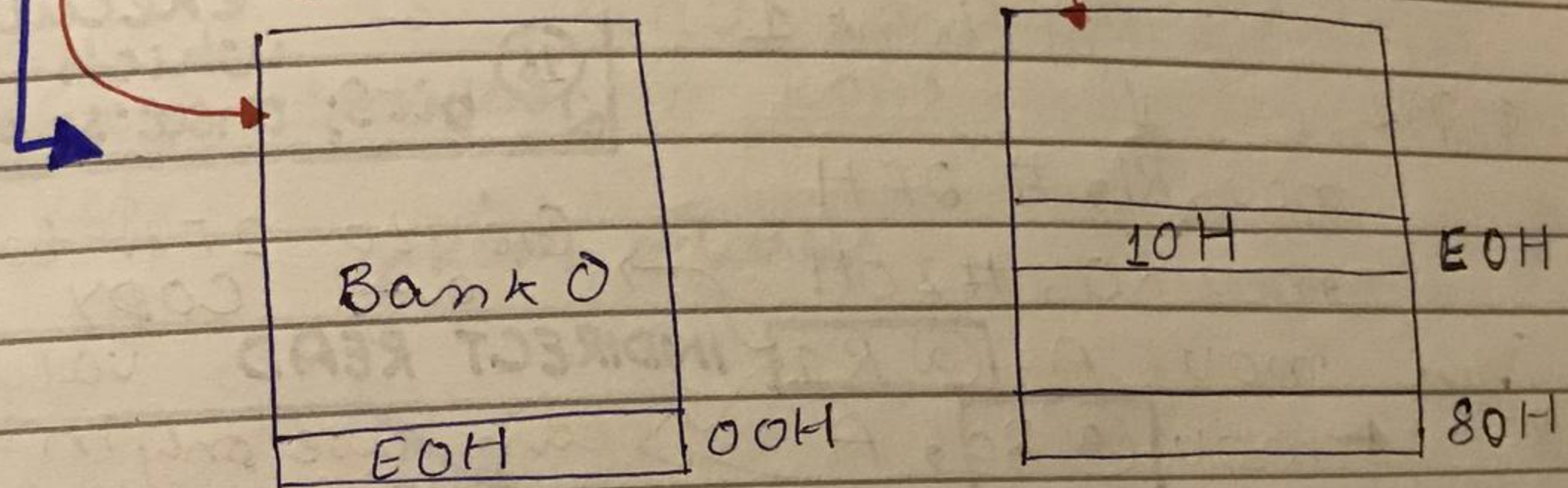
- codes written on 89C51 will also run on 89C52.

Using indirect Addressing mode to use 'EXTRA 128 bytes' of 89C52

(i) `mov 80H, #12H` (for 89C52)
 Q- In which 80H will it go?
 A- SFRs

INDIRECT WRITING

(ii) For accessing Accumulator (89C52)
`[ mov E0H, #12H ]` → SFR (Acc ← 12H) because of Direct address
`[ mov R0, #0E0H ]`
`[ mov @R0, #10H ]`



1
 For 89C52:

`[ mov E0H, #12H ]`
`[ mov R0, #E0H ]`
`[ mov @R0, #10H ]`

both commands will do the same task

READING

`mov 30H, @R0`

extra 128 bytes for 89C52

0x30
0x
30H

e.g. 1

mov A, #0FH
mov R0, #2FH
inc A
mov @R0, A

continued

of bank 1

mov R1, #2FH
mov R0, #20H

mov A, @R1
mov @R0, A

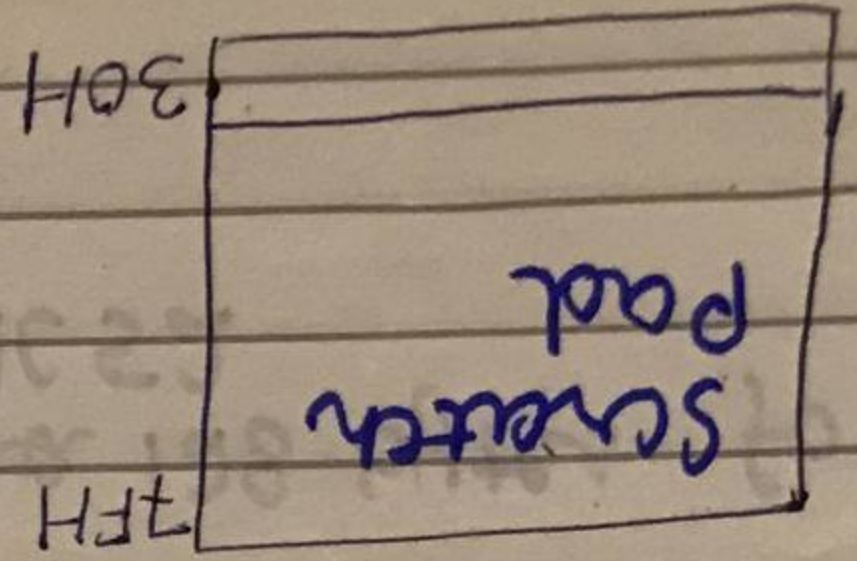
use value of 'R0' as address and copy the value of A on that location.

INDIRECT WRITE

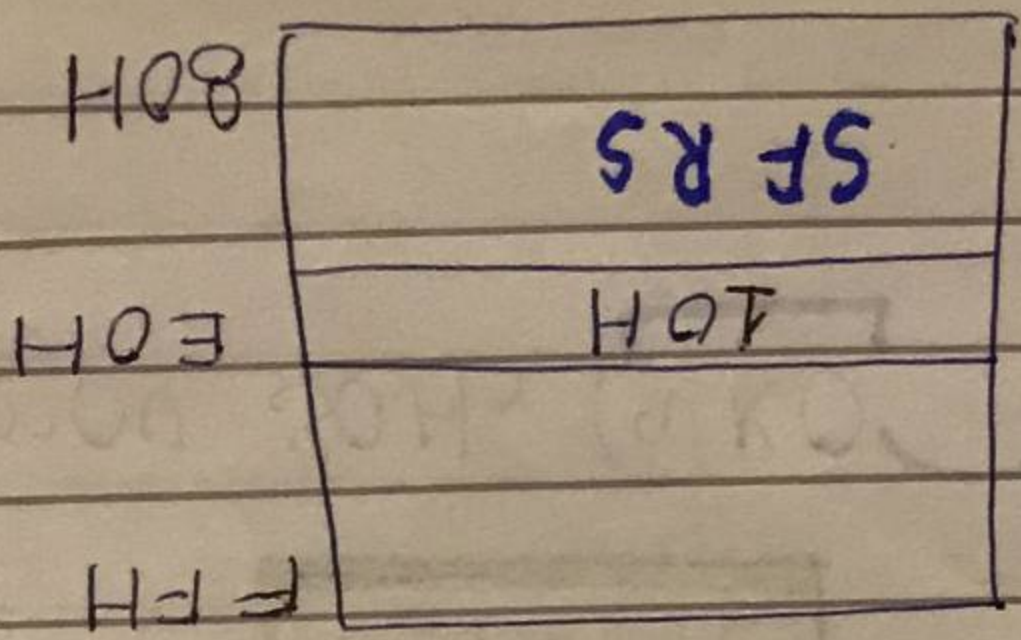
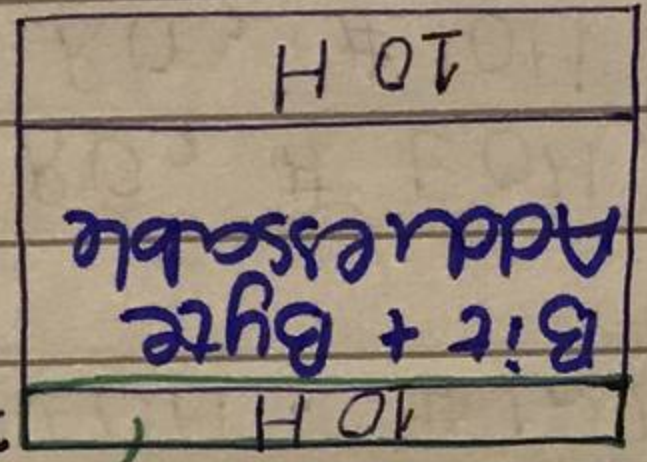
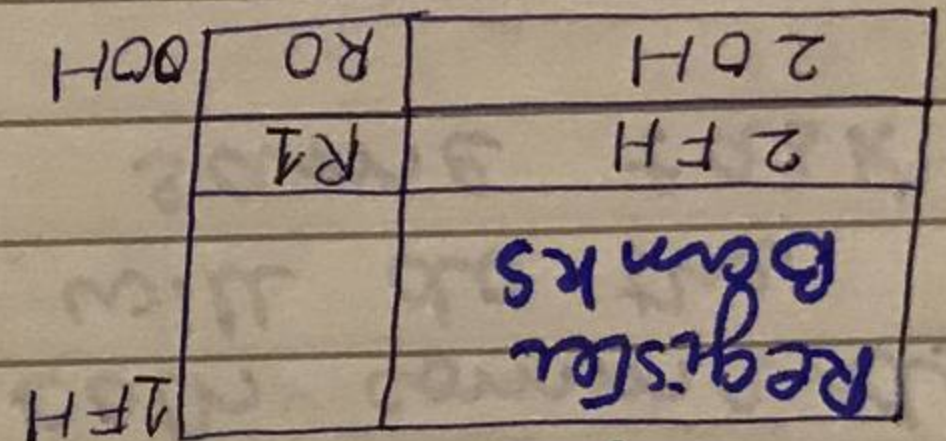
INDIRECT READ

Go to 2FH location and copy its value on A
can use only in it R0 and R1

about 2FH above part.



Bank 0



Program counter (16 bit) holds address of Rom
originate 0x0 Rom address
*you can use external Rom of 64k Bytes to execute the code which needs 16 bit
bits; that's why 16 bit
end

mov A, #30H 2 Bytes
mov B, #2FH
mov R0, B
mov R1, A

PC main hai instruction ki space hai (mostly). 193 bytes ki bhi hai hen

16 bit wide Data Pointer
 don't want to use your RAM, you want to use RAM, to store data that adds up at run-time (from ROM). we need something from ROM. anything

Use want to create a table / arrays / look-up tables in ROM with no-overlapping issues

org 0x0
 1 byte 2 byte
 mov R0, #80H
 mov R1, #2FH
 mov A, @R0
 mov @R1, A
 org 30H /
 org 0x30
 e.g.

0006 mov @R1, A
 0008 Free space
 DPTR will always use the next free location. Since 0008 is free, so it'll use it

(16)

array 1: DB 07H, 7FH, 23, 0001 0101 B
label 0008 0009 000A 000B 15H
define byte

end

For 0th element of array to be accessed
you can use the "Label" in your
runtime program.

mov DPTR, # array1

→ DPTR will start pointing
at the 0th element of
array1 (defined in ROM of uC)

→ kuch aur allow nhe hai.

mov A, #3

0003H

movc A, @ A + DPTR

location of
DPTR = 0008

code
program

0008H
+ 0003H

000BH

now, go to 000B H (location)

and copy its value

on A

A ← 0001 0101 B
15H

away 1: DB 0008 0009 000A 000B 07H, 7FH, 23, 00H 010B

mov A, #0

mov DPTR, # away 1 // not to lose location

mov R1, A // location

move A, @A + DPTR making copy

mov 30H, A R1

inc R1 // k andar 1 para hai

abb.

* mov A, R1 // A k andar 1 hai

movc

31H @A + DPTR

7FH 31H

Stack Pointer (RAM) - SP

8 bit wide

by default, it points at 07H.

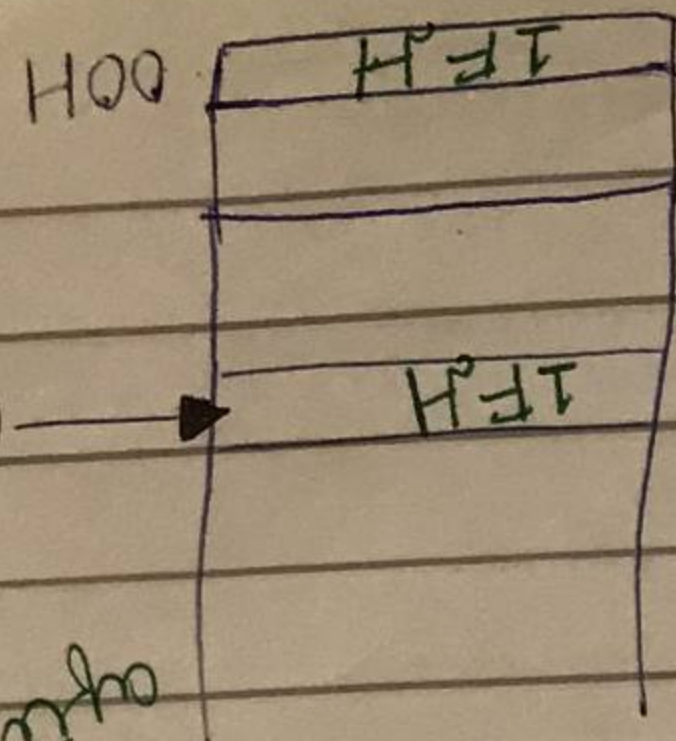
push 08H

(89C51 - pre inc. push) perley data in location

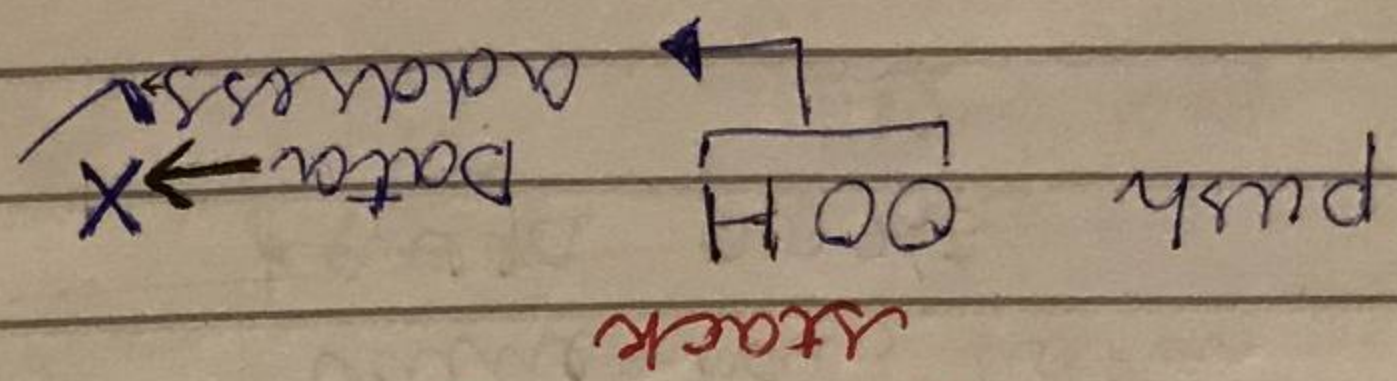
hoga, phir push

hoga hai.

007 initially
after command (inc) 008



copy of 00H on 008 location



08H pay jo value hai, wo push hogi in stack

application for example
2) job kr karke koi karna hai pop

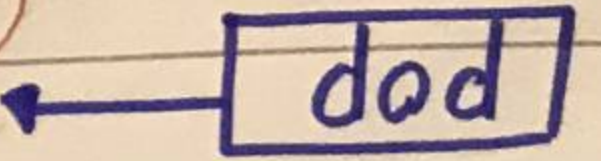
Stack

Advantage: 1) Function calls

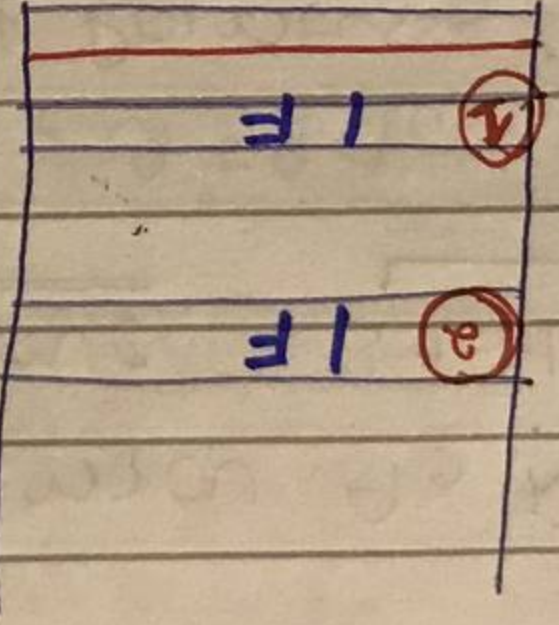
R1 0

A 0

last used location seay (pop kar ke) data copy karaga on the provided location and phir decrement karaga in location.



e.g. pop 30H



08H (before POP command) → 07H → e.g.

e.g.

mov SP, # 30H

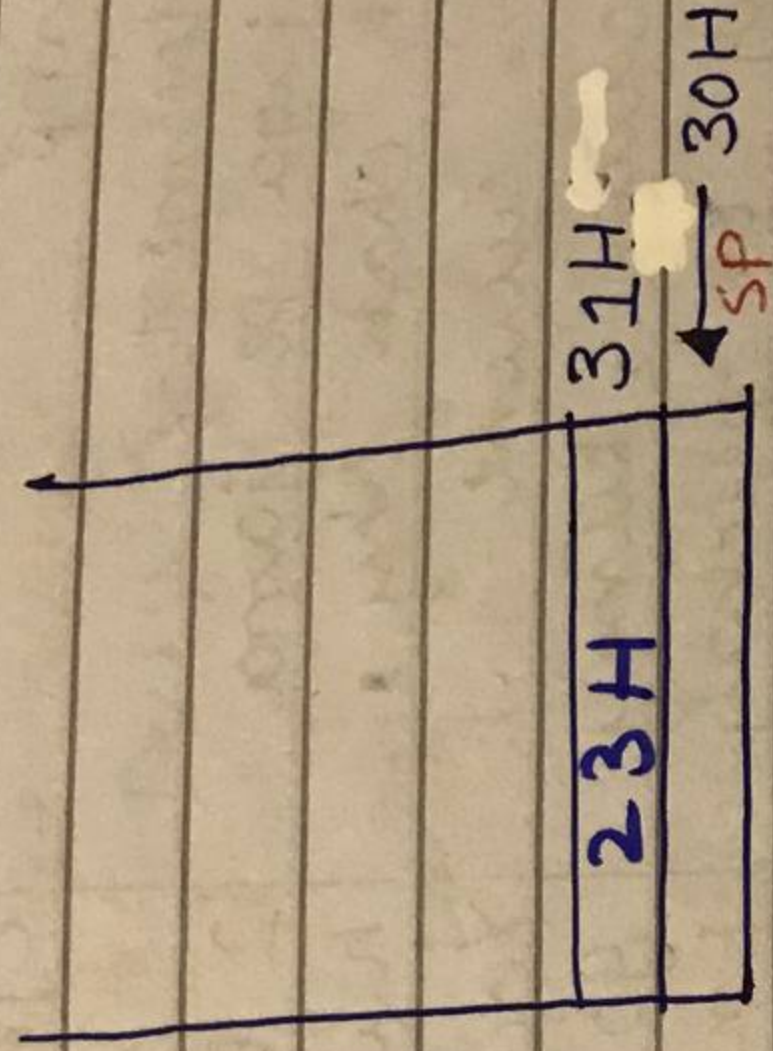
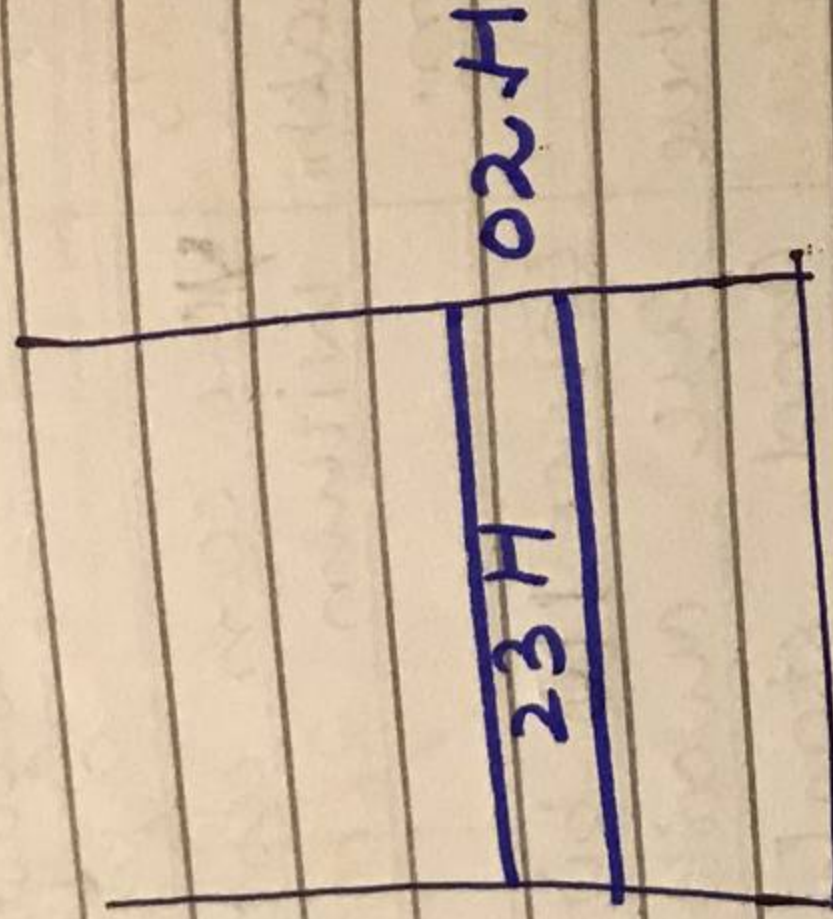
30H location ko point karaga

gh. scratch pad par pay jamaay k liay.

mov SP, 30H // 30H k andar jo value hai, uska karaga point.

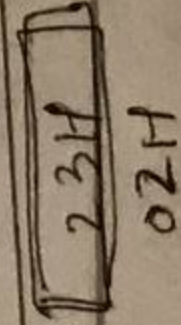
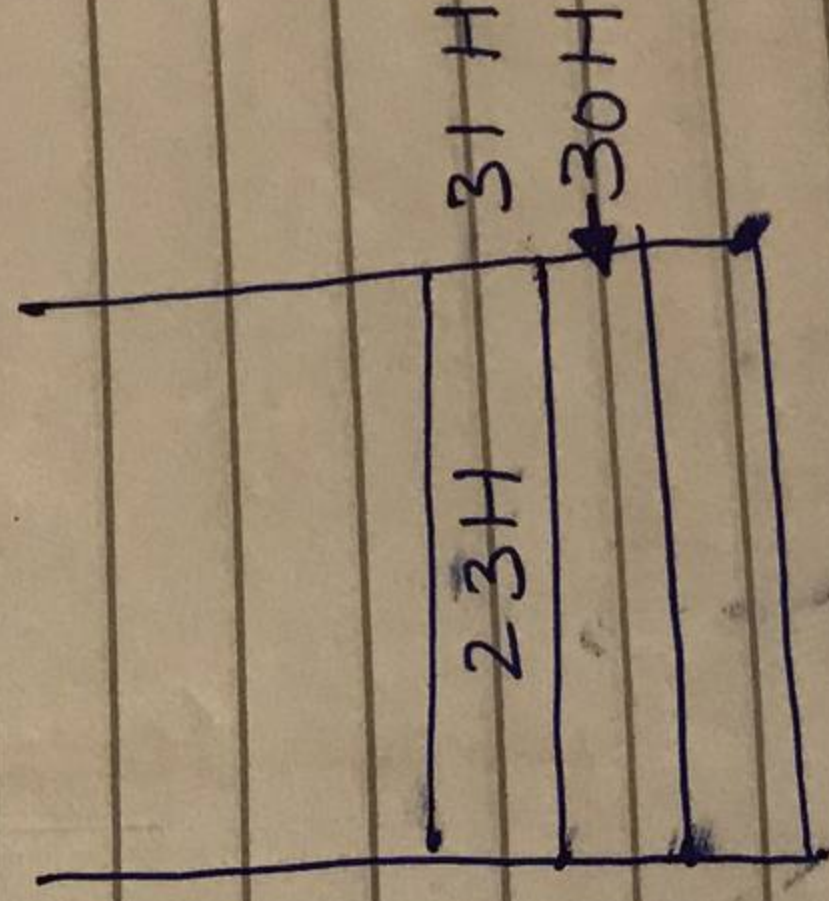
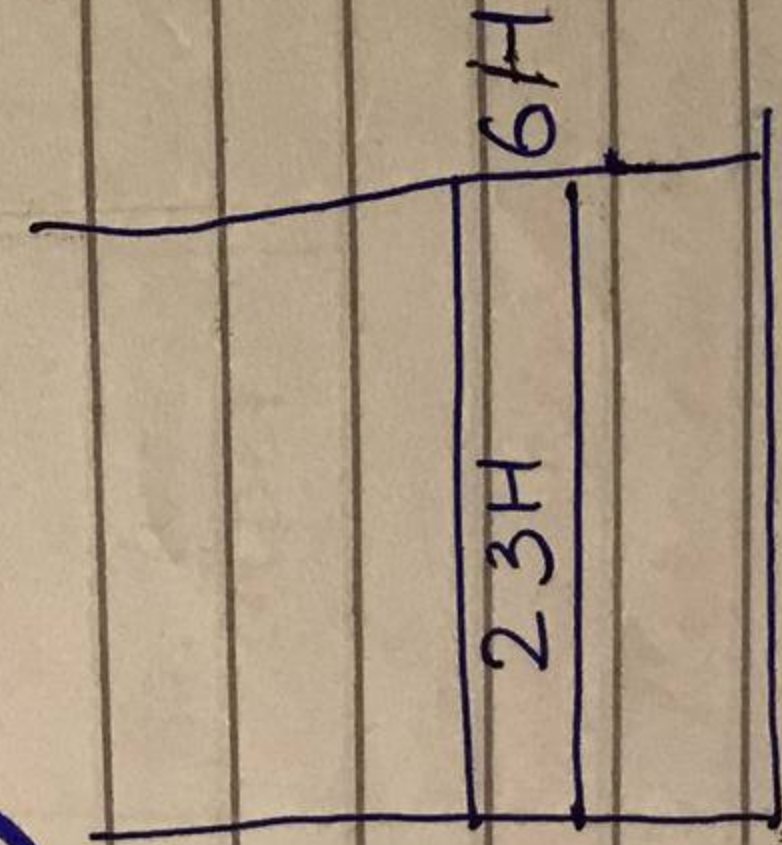
A mein value daal k we
can use Acc * Acc; 0E0H

push
pop



02H location ka data copy kardo 31H
par (push 02H) now pointing
at 31H

pop → copy kar ta, del nhi



SP, 30H

push 02H

31H

pop

(20)

- All conditional jumps are jumps by default.

11-Sept-2017

(You cannot change its nature.)

Unconditional Jump:

Jump without any condition.

- USE LABELS

short jump Sjmp	long jump Ljmp	absolute jump ajmp
displacement=? jis jaga pe jana chah chah shy, from current location, kitnay aagay ya peechay jana hai, DISPLACEMENT- NT batao? [1 byte -128 — +127	PC = 16 bits [2 bytes] rakhi hui hain. You can go anywhere Exact address	You can use it within 2k Bytes. Lower 11 bits are available and starting 5 MSBs are common. • Medium jump

(21)

unconditional jump mixed with subroutine

org 0x0

main: p1.0 → 1

call delay - 1s → 2

jmp main

Function call.

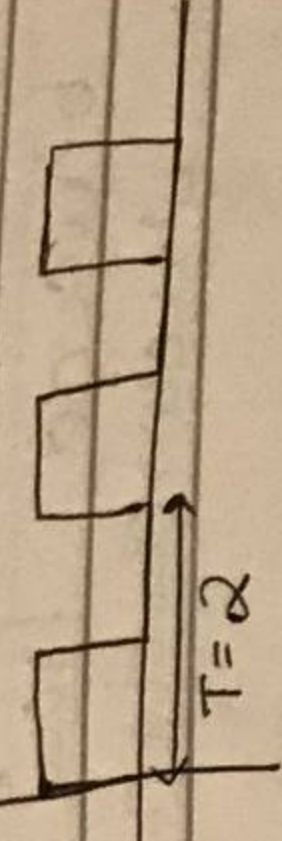
delay 1-s

Yes

pop out 2 values (default)

end.

D.C = 50%



Square wave of $f = 0.5 \text{ Hz}$ store the address

of PC (next Duty cycle: T_{on} of page)

Total time

end.

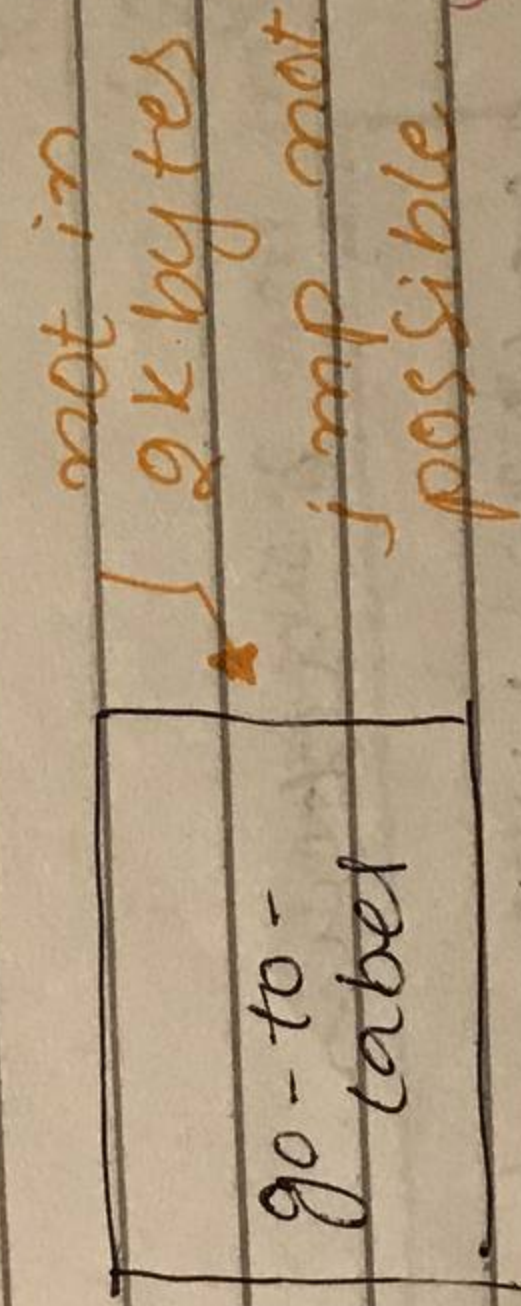
$T = 2$

D.C = 75%

Absolute jump:

org 0x0001

ajmp go-to-label



PUSH PC ko kia tha, POP bhi

PC mein hoga. (All work implicitly by CPU)

• It remembers kis order mein push hua.

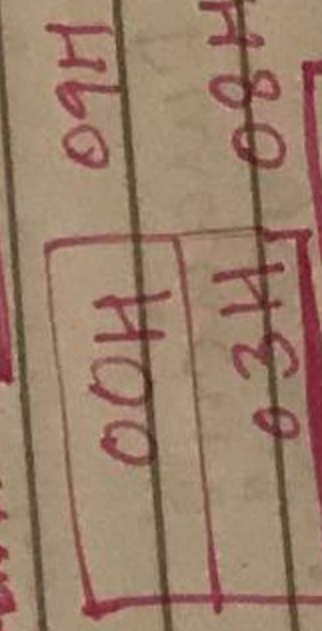
75% of your signal for $T = 2$ is "on"

PC = 0003
MSB LSB

2 pushes

SP → 07H (Default) stack will we 08H, 09H to store the above 2 bytes.

Order: LITTLE ENDIAN



Big ENDIAN

opposite of above

(92) contains code.

RAM - Each location 1 byte
 ROM - 2 bytes

local call

Subroutine: uopis aamay xasta

bitama hai - unconditional jump with a return.

PC pointer should be stored somewhere when function is called so that it comes to the immediate next instruction of "Function call". CPU uses stack for this purpose, implicitly.

a call

absolute call.

label within 2 k Bytes.

call can be made anywhere

64 k Bytes.

no return.

Agar khaali "call" likha hai tou use khudi decide karta hai k long call ya a call.

Duration? machine cycle time? loop 1: 11: 0x10/F

(23)

Delay generation:of 8051 μC

1 MC time

↳ machine cycle

= 12 clock periods

1 clock period = $\frac{1}{f_{osc}}$ f_{osc} ↳ oscillator frequencye.g. $f_{osc} = 4\text{MHz}$

$$MC = \frac{12}{4 \times 10^6} = 3\mu s.$$

Conditional Jumps:

↳ Direct RAM location. Ray kaam karta hai.

D J N Z → Decrement jump if not zero.

(1)

DJNZ Rn, label

Pehlay dec. karay ga,
 phir check karay ga
 k zero tou nhe hai.

Agar '0' hai tou

immediate next inst. ~~data~~ pay chala jayga.

Tab tak value '0' nhe hoti wapis label
 pay chala jayga.

(2) DJNZ, direct, label

Example (1)

clr P3.3 #255

mov R0, #03 → 1 μC

here:

DJNZ R0, here

Set b P3.3

→ 2 μC → for one execution

Time b/w ~~to~~ Resetting $\rightarrow$ P3.3 and clearing

(24) selected crystal (fixed now)

$f_{osc} = 12 \text{ MHz}$, $\Rightarrow MC = 1 \mu\text{C}$

$R0, \#1000H \leftarrow 1 + (2 \times 3) \times 1000H$
 $7 \mu\text{MCS}$ $\rightarrow$ DNZ was executed 255 3 times.

$$1 + (2 \times 255) = 511 \text{ MCS} \times 1 \mu\text{S}$$

Delay 1-5:

MOV R0, #255

here: D-JNZ R0, here $\approx 0.5 \text{ msec}$

MOV R0, #255

here 1: DNZ R0, here 1 $\approx 0.5 \text{ msec}$

50% duty cycle

ret

1m 1m

$$T = 2 \text{m}$$
$$f = 0.5 \text{ Hz}$$

CODE:

ORG 0x0

main:

CPL P3.0

call delay-1msec

(By default all ports are logic 1)

jmp main

delay-1 msec:

use the above 1-msec "Delay" here

ret

end

$$f = \frac{12 \text{ MHz}}{12} = 1 \text{ MHz}$$

(2)

if we want a delay of 10ms,

mov R1, #10

here: call delay = 1 msec

D-JNZ R1, here.

for the generation 1 sec Delay:

we need more D-JNS

$$1 \text{ MC time} = 1 \mu \text{ sec}$$

$$\frac{1 \text{ sec}}{1 \mu} = \frac{1 \text{ M}}{2 \times 255} = 1960$$

Total machine cycles: executions

7.68

1960 / 255

Less than 255.

No more D-JNS

mov R1, #2

mov R0, #255

Reload - R0:

[2MC x 255] * 2

Inner most loop

here: D-JNZ R0, here

D-JNZ R1, reload-R0

4, 1, 0

0